

Chapter 17: Advanced Uses of Pointers

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Note: These slides are updated based on original notes
from Raj Sunderraman, Michael Weeks, and Yuan Long

Dynamic Storage Allocation

- C's data structures, including arrays, are normally fixed in size.
- Fixed-size data structures can be a problem, since we're forced to choose their sizes when writing a program.
- Fortunately, C supports ***dynamic storage allocation***: the ability to allocate storage during program execution.
- Using dynamic storage allocation, we can design data structures that grow (and shrink) as needed.

Memory Allocation Functions

- Dynamic storage allocation is used most often for strings, arrays, and structures.
- The `<stdlib.h>` header declares three memory allocation functions:
 - `malloc`—Allocates a block of memory but doesn't initialize it.
 - `calloc`—Allocates a block of memory and clears it. (sets 0)
 - `realloc`—Resizes a previously allocated block of memory.
- These functions return a value of type `void *` (a “generic” pointer).

Null Pointers

- An example of testing `malloc`'s return value:

```
p = malloc(10000);  
if (p == NULL) {  
    /* allocation failed; take appropriate action */  
}
```

- `NULL` is a macro (defined in various library headers) that represents the null pointer.
- Some programmers combine the call of `malloc` with the `NULL` test:

```
if ((p = malloc(10000)) == NULL) {  
    /* allocation failed; take appropriate action */  
}
```

Using malloc to Allocate Memory for a String

- Prototype for the `malloc` function:

```
void *malloc(size_t size);
```
- `malloc` allocates a block of `size` bytes and returns a pointer to it.
- `size_t` is an unsigned integer type defined in the library.

Using malloc to Allocate Memory for a String

- A call of `malloc` that allocates memory for a string of `n` characters:

```
p = malloc(n + 1);
```

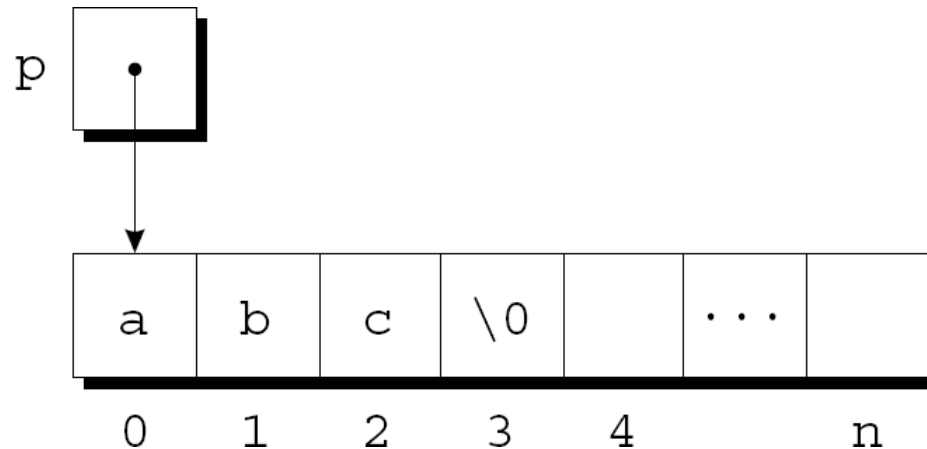
`p` is a `char *` variable.

- Each character requires one byte of memory; adding 1 to `n` leaves room for the null character.
- Some programmers prefer to cast `malloc`'s return value, although the cast is not required:

```
p = (char *) malloc(n + 1);
```

Using malloc to Allocate Memory for a String

- Calling `strcpy` is one way to initialize this array:
- `strcpy(p, "abc");`
- The first four characters in the array will now be a, b, c, and `\0`:



Using `malloc` to Allocate Storage for an Array

- Suppose a program needs an array of n integers, where n is computed during program execution.
- We'll first declare a pointer variable:
 - `int *a;`
- Once the value of n is known, the program can call
- `malloc` to allocate space for the array:
 - `a = malloc(n * sizeof(int));`
- Always use the `sizeof` operator to calculate the amount of space required for each element.

Example – Primitive integer

```
int n, i, *ptr, sum = 0;

printf("Enter number of elements: ");
scanf("%d", &n);

ptr = (int*) malloc(n * sizeof(int));

printf("Enter elements: ");
for(i = 0; i < n; ++i) {
    scanf("%d", ptr + i);
    sum += *(ptr + i);
}
```

Example - structure

```
struct rec
{
    char LastName[41];
    char FirstName[41];
};
```

```
struct rec      *r;
r = (struct rec *) malloc(sizeof(struct rec));
```

String input – a caution

```
struct course {
    int marks;
    char subject[30];
};

int main() {
    struct course *ptr;

    /* Clear the buffer */
    fflush(stdin);

    // Memory allocation for a structure
    ptr = (struct course *)malloc(sizeof(struct course));

    printf("Enter subject name:\n");
    /* fgets allows to include spaces but also includes \n */
    fgets(ptr->subject, sizeof(ptr->subject), stdin);

    /* remove trailing \n character */
    size_t len = strlen(ptr->subject);
    if (len > 0 && (ptr->subject[len - 1] == '\n') {
        ptr->subject[len - 1] = '\0';
    }
    printf("Enter marks:");
    scanf("%d", &ptr->marks);
}
```